```
clear;
close all;
format shortG;
```

Function of Risk Measures and VAR and CVAR at 95%, 97% and 99%

```
% Calculates risk statistics and Value at Risk (VaR) for a single asset.
function RiskStats = calculateRiskStats(LogReturns, AssetName)
    LogReturns = LogReturns(~isnan(LogReturns));
    nReturns = length(LogReturns);

    % Statistic in decimals
    mean_ret = mean(LogReturns);
    std_dev = std(LogReturns);
    variance = var(LogReturns);
    skew_val = skewness(LogReturns);
    kurt_val = kurtosis(LogReturns);

    RiskStats = table({AssetName}, nReturns, mean_ret*100, std_dev*100,
variance*100, kurt_val, skew_val, ...
                      'VariableNames', {'Asset Name', 'Number_of_Returns', 'Mean
(%)', 'StdDev (%)', ...
                                        'Variance (%)', 'Kurtosis', 'Skewness'});

    confidence_levels = [0.95, 0.97, 0.99];

    for j = 1:length(confidence_levels)
        z = norminv(confidence_levels(j));
        VaR = -(mean_ret + z * std_dev);
        phi_z = exp(-0.5 * z^2) / sqrt(2*pi);
        CVaR = -(mean_ret + std_dev * (phi_z / (1 - confidence_levels(j))));

        RiskStats{1, ['VaR_' num2str(round(confidence_levels(j)*100))]} = VaR * 100;
        RiskStats{1, ['CVaR_' num2str(round(confidence_levels(j)*100))]} = CVaR *
100;
    end

    RiskStats{:, 2:end} = round(RiskStats{:, 2:end}, 4);
    disp(RiskStats);
end
```

Histogram with density curve and Q-Q plot

```
% This function generates a histogram with density curve and a Q-Q plot for a
single asset.
function generatePlotsForAsset(LogReturns, AssetName)
    % Histogram Plot
    figHist = figure;
    histogram(LogReturns, 30, 'Normalization', 'pdf');
```

```
    hold on;
    [f, xi] = ksdensity(LogReturns);
    plot(xi, f, 'LineWidth', 2, 'Color', 'r');
    hold off;
    title(['Histogram and Density Curve of Log Returns for ', AssetName]);
    xlabel('Log Returns');
    ylabel('Density');

    % Q-Q Plot
    figQQ = figure;
    qqplot(LogReturns);
    title(['Q-Q Plot of Log Returns for ', AssetName]);
end
```

Function of iid Proprierties Test (Autocorrelation and Normality) - (Ljung-Box and D'Agostino-Pearson)

```
% IID PROPERTIES TEST (Autocorrelation and Normality) - (LJUNG-BOX and D'AGOSTINO-
PEARSON)
% Checks whether there are significant autocorrelations at any lag, which would
indicate that the data is not independent.
% Tests whether the data follows a normal distribution and checks for skewness and
kurtosis.
function [autocorrResults, turningPointResults] = testIIDProperties(LogReturns,
AssetName)
    autocorrResults = table('Size', [1, 8], 'VariableTypes', {'double', 'double',
'double', 'double', 'double', 'cell', 'cell', 'cell'}, ...
        'VariableNames', {'LjungBox_pValue', 'LjungBox_QStat', 'LjungBox_CritVal',
'Normality_pValue', 'JarqueBera_pValue', ...
                          'AutocorrelationConclusion', 'NormalityConclusion_JB',
'NormalityConclusion_DA'}, ...
        'RowNames', {AssetName});

    % Ljung-Box test for autocorrelations
    [~, pVal, QStat, critVal] = lbqtest(LogReturns, 'Lags', 10);
    autocorrResults{1, 'LjungBox_pValue'} = pVal;
    autocorrResults{1, 'LjungBox_QStat'} = QStat;
    autocorrResults{1, 'LjungBox_CritVal'} = critVal;
    if pVal < 0.05
        autocorrConclusion = 'Autocorrelated';
    else
        autocorrConclusion = 'No-Autocorrelated';
    end

    % Jarque-Bera test for normality
    [~, pJB] = jbtest(LogReturns);
    autocorrResults{1, 'JarqueBera_pValue'} = pJB;
    if pJB < 0.05
```

```
        normalityConclusionJB = 'Not Normal';
    else
        normalityConclusionJB = 'Normal';
    end

    % D'Agostino-Pearson test for normality
    n = length(LogReturns);
    skewVal = skewness(LogReturns);
    kurtVal = kurtosis(LogReturns);
    SES = sqrt(6 * n * (n - 1) / ((n - 2) * (n + 1) * (n + 3)));
    ZS = skewVal / SES;
    SEK = 2 * SES * sqrt((n^2 - 1) / ((n - 3) * (n + 5)));
    ZK = (kurtVal - 3) / SEK;
    K2 = ZS^2 + ZK^2;

    pValueDA = 1 - chi2cdf(K2, 2);
    autocorrResults{1, 'Normality_pValue'} = pValueDA;
    if pValueDA < 0.05
        normalityConclusionDA = 'Not Normal';
    else
        normalityConclusionDA = 'Normal';
    end
    autocorrResults{1, 'AutocorrelationConclusion'} = {autocorrConclusion};
    autocorrResults{1, 'NormalityConclusion_JB'} = {normalityConclusionJB};
    autocorrResults{1, 'NormalityConclusion_DA'} = {normalityConclusionDA};

    % Turning Point Test for iid
    % For the Turning Point test, we calculate the number of turning points in the
data, the expected number of turning points,
    % the z-score, and the p-value for the test.
    turningPointResults = table('Size', [1, 6], 'VariableTypes', {'double',
'double', 'double', 'double', 'logical', 'cell'}, ...
        'VariableNames', {'TurningPoints', 'ExpectedTurningPoints',
'ZScoreTurningPoints', 'PValueTurningPoints', 'IsIID', 'IIDConclusion'}, ...
        'RowNames', {AssetName});

    % Calculate turning points
    diffData = diff(LogReturns);
    turningPoints = sum(diffData(1:end-1) .* diffData(2:end) < 0);
    expectedTurningPoints = (n - 1) / 2;
    varianceTurningPoints = (16 * n - 29) / 90;
    zScore = (turningPoints - expectedTurningPoints) /
sqrt(varianceTurningPoints);
    pValue = 2 * (1 - normcdf(abs(zScore)));
    if pValue < 0.05
        Turning_points_Conclusion = 'Not iid';
    else
        Turning_points_Conclusion = 'iid';
    end
```

```
    % Add the results to the table
    turningPointResults{1, 'TurningPoints'} = turningPoints;
    turningPointResults{1, 'ExpectedTurningPoints'} = expectedTurningPoints;
    turningPointResults{1, 'ZScoreTurningPoints'} = zScore;
    turningPointResults{1, 'PValueTurningPoints'} = pValue;
    turningPointResults{1, 'IIDConclusion'} = {Turning_points_Conclusion};

    % Display the results
    disp(autocorrResults);
    disp(turningPointResults);
end
```

Graphic ACF and PACF

```
function plotACFandPACF(LogReturns, AssetName, numLags)
    figure('Name', ['ACF & PACF for ', AssetName]);

    % ACF
    subplot(2,1,1);
    autocorr(LogReturns, 'NumLags', numLags);
    title(['ACF for ', AssetName], 'FontSize', 12);
    xlabel('Lag', 'FontSize', 10);
    ylabel('Autocorrelation', 'FontSize', 10);
    grid on;

    % PACF
    subplot(2,1,2);
    hold on;
    parcorr(LogReturns, 'NumLags', numLags);
    title(['PACF for ', AssetName], 'FontSize', 12);
    xlabel('Lag', 'FontSize', 10);
    ylabel('Partial Autocorrelation', 'FontSize', 10);
    grid on;
    sgtitle(['Model Identification (ACF & PACF) for ', AssetName], 'FontSize', 14,
'FontWeight', 'bold');
end
```

AR-GARCH

AR-GARCH (t-distribution)

```
function [ADFResults, AR_GARCHResults, stdResiduals, ResidualDiagnostics,
AdditionalDiagnostics, VaR_CVaR, EstModel, condVar] =
AR_GARCH_Model_SingleAsset_tStudents(LogReturns, AssetName, ConfidenceLevels)
%% AR-GARCH (t-distribution) con VaR/CVaR coerenti

    if nargin < 3
```

```
        ConfidenceLevels = [0.95, 0.97, 0.99]; % default
    end

    %% 1. ADF Test for Stationarity
    [~, pValue, stat, ~, cValue] = adftest(LogReturns);
    if ~isempty(cValue) && numel(cValue) >= 2
        criticalVal = cValue(2);  % livello 5%
    else
        warning('ADF critical value not available, using default -2.86');
        criticalVal = -2.86;
    end
    stationarityConclusion = ternary(pValue < 0.05, "Stationary", "Non-Stationary");
    ADFResults = table(pValue, stat, criticalVal, stationarityConclusion, ...
        'VariableNames',
{'pValue','TestStatistic','CriticalValue_5%','StationarityConclusion'}, ...
        'RowNames', {AssetName});

    %% 2. AR(1)-GARCH(1,1) Estimation with t-Distribution Innovations
    try
        varianceModel = garch(1,1);
        model = arima('Constant',
NaN,'ARLags',1,'Variance',varianceModel,'Distribution','t');
        [EstModel, EstCov] = estimate(model, LogReturns, 'Display','off');
        [residuals, condVar] = infer(EstModel, LogReturns);
    catch ME
        warning(['AR-GARCH estimation failed for ', AssetName, '. Error: ',
ME.message]);
        [ADFResults, AR_GARCHResults, stdResiduals, ResidualDiagnostics,
AdditionalDiagnostics, VaR_CVaR, EstModel, condVar] = deal([]);
        return;
    end
    stdResiduals = residuals ./ sqrt(condVar);

    %% 3. Extract Model Parameters
    dof = EstModel.Distribution.DoF;
    params = [EstModel.AR{1}, EstModel.Variance.ARCH{1},
EstModel.Variance.GARCH{1}, ...
              EstModel.Variance.Constant, EstModel.Constant, dof];
    se = sqrt(diag(EstCov));
    persistence = params(2) + params(3);

    AR_GARCHResults = table({AssetName}, round(params(1),5), round(params(2),5), ...
        round(params(3),5), round(params(4),5), round(params(5),5),
round(params(6),2), ...
        round(se(1),5), round(se(2),5), round(se(3),5), round(se(4),5),
round(se(5),5), round(se(6),2), ...
        persistence, ...
        'VariableNames',
{'AssetName','AR1','ARCH1','GARCH1','Omega','Constant','DoF', ...
```

```
'SE_AR1','SE_ARCH1','SE_GARCH1','SE_Omega','SE_Constant','SE_DoF','ARCHplusGARCH'});

    %% 4. Residual Diagnostics
    [~, lbq_p_resid] = lbqtest(stdResiduals,'Lags',10);
    [~, lbq_p_sqresid] = lbqtest(stdResiduals.^2,'Lags',10);
    tpResult = turnpointtest(stdResiduals);
    iidConclusion = ternary(tpResult.pValue<0.05,"non-i.i.d.","i.i.d.");

    ResidualDiagnostics = table(lbq_p_resid, lbq_p_sqresid, tpResult.pValue,
iidConclusion, ...
        'VariableNames',
{'LjungBox_Resids_pVal','LjungBox_ResidsSq_pVal','TurningPoint_pVal','iidConclusion'
}, ...
        'RowNames',{AssetName});

    %% 5. Additional Diagnostics
    [~, pValueARCH, statARCH, critARCH] = archtest(stdResiduals,'Lags',5);
    [~, pValueJB, statJB] = jbtest(stdResiduals); % solo informativo
    AdditionalDiagnostics = table(pValueARCH, statARCH, critARCH, pValueJB, statJB,
...
        'VariableNames',
{'ARCH_pValue','ARCH_TestStatistic','ARCH_CriticalValue_5%','JB_pValue','JB_TestStat
istic'}, ...
        'RowNames',{AssetName});

    %% 6. VaR and CVaR coerenti con distribuzione t stimata
    n = length(LogReturns);
    sigma_t = sqrt(condVar); % volatilità condizionale
    nu = EstModel.Distribution.DoF;
    mu = EstModel.Constant;  % media AR-GARCH

    VaR_CVaR = table('Size',
[1,length(ConfidenceLevels)*2],'VariableTypes',repmat("double",1,length(ConfidenceLe
vels)*2));
    varNames = cell(1,length(ConfidenceLevels)*2);

    for k = 1:length(ConfidenceLevels)
        cl = ConfidenceLevels(k);
        % VaR t-Student
        t_quant = tinv(cl, nu);
        VaR = -(mu + sigma_t(end)*t_quant*sqrt((nu-2)/nu));
        % CVaR t-Student (Expected Shortfall)
        pdf_t = @(x) tpdf(x,nu);
        ES = -(mu + sigma_t(end) * nu/(nu-1) * tpdf(t_quant,nu)/(1-cl));

        VaR_CVaR{1,(k-1)*2+1} = VaR*100;       % VaR in percent
        VaR_CVaR{1,(k-1)*2+2} = ES*100;        % CVaR in percent
        varNames{(k-1)*2+1} = ['VaR_' num2str(round(cl*100))];
        varNames{(k-1)*2+2} = ['CVaR_' num2str(round(cl*100))];
```

```
    end
    VaR_CVaR.Properties.VariableNames = varNames;
    VaR_CVaR.Properties.RowNames = {AssetName};
end

%%  Helper functions
function result = turnpointtest(data)
    n = length(data);
    if n < 3, error("Data series must have at least 3 observations."); end
    tp = sum((data(2:end-1) > data(1:end-2) & data(2:end-1) > data(3:end)) | ...
             (data(2:end-1) < data(1:end-2) & data(2:end-1) < data(3:end)));
    expectedTP = 2*(n-2)/3;
    varianceTP = (16*n-29)/90;
    z = (tp-expectedTP)/sqrt(varianceTP);
    p = 2*(1-normcdf(abs(z)));
    result = struct('pValue',p,'TurningPoints',tp,'ExpectedTP',expectedTP);
end

function out = ternary(cond,valTrue,valFalse)
    if cond, out = valTrue; else, out = valFalse; end
end
```

Mean - Excess plot

```
function plotMeanExcess(LogReturns, outputDir, AssetName, nPoints)
    if nargin < 4
        nPoints = 100; % default number of points
    end

    if ~exist(outputDir, 'dir')
        mkdir(outputDir);
    end

    % Creazione dei livelli di soglia
    u_values = linspace(min(LogReturns), max(LogReturns), nPoints);
    meanExcess = zeros(size(u_values));

    % Calcolo dell'eccedenza media
    for i = 1:length(u_values)
        u = u_values(i);
        exceedances = LogReturns(LogReturns > u);
        if ~isempty(exceedances)
            meanExcess(i) = mean(exceedances - u);
        else
            meanExcess(i) = NaN; % se non ci sono superamenti
        end
    end
```

```
    % Plot
    fig = figure('Visible','on');
    plot(u_values, meanExcess, 'LineWidth', 2);
    xlabel('Threshold u');
    ylabel('Mean Excess');
    title(['Mean Excess Plot - ' AssetName]);
    grid on;

    % Salvataggio
    saveas(fig, fullfile(outputDir, ['MeanExcess_' AssetName '.png']));
    disp(['Mean Excess plot salvato in: ' fullfile(outputDir, ['MeanExcess_'
AssetName '.png'])]);
end
```

Threshold selection

```
function threshold_selection(stdResiduals, AssetName, thresholds, p, outDir,
leftTail)

    if nargin < 4, p = [0.99]; end
    if nargin < 5, outDir = pwd; end
    if nargin < 6, leftTail = false; end
    if ~exist(outDir, 'dir'), mkdir(outDir); end

    data = stdResiduals(:);
    data = data(~isnan(data));

    if leftTail
        data = -data; % invertiamo per modellare la coda sinistra
    end

    nU = numel(thresholds);
    nP = numel(p);
    meanExcess = nan(nU,1);
    VaR_est = nan(nU,nP);
    ES_est = nan(nU,nP);
    AD_stat = nan(nU,1);
    CVM_stat = nan(nU,1);

    for i = 1:nU
        u = prctile(data, thresholds(i));
        exceedances = data(data > u) - u; % sempre >0 dopo inversione

        if numel(exceedances) > 50
            % Mean Excess
            meanExcess(i) = mean(exceedances);

            % Fit GPD
```

```
            [parmhat,~] = gpfit(exceedances);
            xi = parmhat(1); sigma = parmhat(2);

            % VaR e ES per ciascun livello p
            for j = 1:nP
                q = 1 - (1-p(j)) * (length(data)/length(exceedances));
                arg = (q^(-xi))-1;
                if isreal(arg) && arg > -1e-8
                    VaR_est(i,j) = u + sigma/xi * arg;
                    if xi < 1
                        ES_est(i,j) = (VaR_est(i,j)+sigma-xi*u)/(1-xi);
                    else
                        ES_est(i,j) = NaN;
                    end
                else
                    VaR_est(i,j) = NaN;
                    ES_est(i,j) = NaN;
                end
            end

            % Goodness-of-Fit (manuale)
            [AD, CVM] = gpd_gof(exceedances, xi, sigma);
            AD_stat(i) = AD;
            CVM_stat(i) = CVM;
        end
    end

    % === 1. Mean Excess Plot ===
    fig1 = figure('Visible','off');
    plot(thresholds, meanExcess,'b-o','LineWidth',1.2);
    title(['Mean Excess Plot - ' AssetName]);
    xlabel('Quantile threshold (%)'); ylabel('Mean Excess');
    grid on;
    saveas(fig1, fullfile(outDir, [AssetName '_MEPlot.png']));
    close(fig1);

    % === 2. Stability Plot (VaR e ES) ===
    fig2 = figure('Visible','off');
    hold on;
    colors = lines(nP);
    for j = 1:nP
        plot(thresholds, VaR_est(:,j),'--','Color',colors(j,:),'LineWidth',1.2);
        plot(thresholds, ES_est(:,j),'-','Color',colors(j,:),'LineWidth',1.2);
    end
    title(['Stability Plot (VaR & ES) - ' AssetName]);
    xlabel('Quantile threshold (%)'); ylabel('Value');
    legendStrings = [strcat("VaR-",string(p)), strcat("ES-",string(p))];
    legend(legendStrings,'Location','best'); grid on;
    saveas(fig2, fullfile(outDir, [AssetName '_StabilityPlot.png']));
    close(fig2);
```

```
    % === 3. Goodness-of-Fit Plot ===
    fig3 = figure('Visible','off');
    plot(thresholds, AD_stat,'m-o','LineWidth',1.2); hold on;
    plot(thresholds, CVM_stat,'c-o','LineWidth',1.2);
    title(['Goodness-of-Fit - ' AssetName]);
    xlabel('Quantile threshold (%)'); ylabel('Statistic');
    legend('Anderson-Darling','CvM','Location','best'); grid on;
    saveas(fig3, fullfile(outDir, [AssetName '_GoF.png']));
    close(fig3);

    disp(['📁 Grafici salvati in: ' outDir]);
end

% === Funzione di supporto per GoF (AD e CvM) ===
function [AD_stat, CVM_stat] = gpd_gof(exceedances, xi, sigma)
    n = numel(exceedances);
    if n < 30
        AD_stat = NaN; CVM_stat = NaN; return;
    end

    % Trasforma i dati in U(0,1) usando la CDF della GPD
    U = cdf('GeneralizedPareto', exceedances, xi, sigma, 0);
    U = sort(U);

    % Cramér–von Mises
    i = (1:n)';
    CVM_stat = sum((U - (2*i-1)/(2*n)).^2) + 1/(12*n);

    % Anderson–Darling
    AD_stat = -n - mean((2*i-1).*(log(U) + log(1 - U(end:-1:1))));
end
```

EVT

EVT with test to improve the affidability - goodness-of-fit, bootstrap CI, rolling window, backtesting

```
function EVT_Results = EVT_POT_Model_Simplified(LogReturns, AssetName,
thresholdLevels, opts)

    if nargin < 4, opts = struct(); end
    if ~isfield(opts,'leftTail'), opts.leftTail = false; end
    if ~isfield(opts,'gofSimulations'), opts.gofSimulations = 1000; end

    N = numel(LogReturns);
    OutTable = table();
```

```
    % Se modello perdite, inverti il segno (operiamo sempre sul "right tail")
    if opts.leftTail
        dataForEVT = -LogReturns(:);
    else
        dataForEVT = LogReturns(:);
    end

    for i = 1:numel(thresholdLevels)
        q = thresholdLevels(i);
        % Threshold (quantile)
        u = quantile(dataForEVT, q);
        exceedances = dataForEVT(dataForEVT > u) - u;
        Nu = numel(exceedances);

        if Nu < 10
            warning('Superamenti troppo pochi per q = %.2f - %s (Nu=%d).', q,
AssetName, Nu);
        end

        % Stima GPD
        xi = NaN; sigma = NaN;
        try
            ph = gpfit(exceedances);
            xi = ph(1); sigma = ph(2);
        catch
            warning('Errore GPD per q = %.2f', q);
        end

        % VaR e CVaR
        if ~isnan(xi)
            p_u = Nu / N;
            if abs(xi) > 1e-8
                VaR = u + (sigma/xi)*((( (1 - q) / p_u ).^(-xi)) - 1);
                if xi < 1
                    CVaR = (VaR + (sigma - xi*(VaR - u))/(1 - xi));
                else
                    CVaR = Inf;
                end
            else
                VaR = u - sigma * log((1 - q) / p_u);
                CVaR = VaR + sigma;
            end
        else
            VaR = NaN; CVaR = NaN;
        end

        % Goodness-of-fit
        if ~isnan(xi)
            [AD_stat, CvM_stat, AD_pval, CvM_pval] = computeADCvM_pval(exceedances,
xi, sigma, 0, opts.gofSimulations);
```

```
        else
            AD_stat = NaN; CvM_stat = NaN; AD_pval = NaN; CvM_pval = NaN;
        end

        % Kupiec test
        if ~isnan(VaR)
            [kupiec_p, kupiec_stat, hitRate] = kupiecTest_original(LogReturns, VaR,
q, opts.leftTail);
        else
            kupiec_p = NaN; kupiec_stat = NaN; hitRate = NaN;
        end

        % Append to table
        newRow = table(q, u, Nu, xi, sigma, VaR, CVaR, AD_stat, CvM_stat, AD_pval,
CvM_pval, ...
            kupiec_p, hitRate, kupiec_stat, ...
            'VariableNames',
{'ThresholdLevel','Threshold_u','NumExceedances','xi','sigma','VaR','CVaR', ...
                             'AD_stat','CvM_stat','AD_pValue','CvM_pValue', ...
                             'Kupiec_pValue','ObservedHitRate','Kupiec_Stat'});
        OutTable = [OutTable; newRow];
    end

    EVT_Results.OutTable = OutTable;
    fprintf('Risultati EVT POT per asset: %s\n', AssetName);
    disp(OutTable);
end

%% --- Subfunctions necessarie ---

function [AD_stat, CvM_stat, AD_pval, CvM_pval] = computeADCvM_pval(data, xi,
sigma, theta, nSim)
    if nargin < 5 || isempty(nSim), nSim = 1000; end
    data = sort(data(:));
    n = numel(data);
    F_est = gp_cdf(data, xi, sigma, theta);
    i = (1:n)';

    AD_stat = -n - sum((2*i-1).*(log(F_est)+log(1-flipud(F_est))))/n;
    CvM_stat = sum((F_est - ((i-0.5)/n)).^2) + 1/(12*n);

    % bootstrap simulazioni
    simAD = nan(nSim,1);
    simCvM = nan(nSim,1);
    for k = 1:nSim
        simSample = gprnd(xi, sigma, theta, n, 1);
        simSample = sort(simSample);
        Fsim = gp_cdf(simSample, xi, sigma, theta);
        simAD(k) = -n - sum((2*i-1).*(log(Fsim)+log(1-flipud(Fsim))))/n;
        simCvM(k) = sum((Fsim - ((i-0.5)/n)).^2) + 1/(12*n);
```

```
    end
    AD_pval = mean(simAD >= AD_stat);
    CvM_pval = mean(simCvM >= CvM_stat);
end

function F = gp_cdf(x, xi, sigma, theta)
    x = (x - theta)./sigma;
    if abs(xi) > 1e-12
        inner = 1 + xi.*x;
        inner(inner <= 0) = eps;
        F = 1 - inner.^(-1./xi);
    else
        F = 1 - exp(-x);
    end
    F = max(0, min(1, F));
end

function [pValue, LRstat, hitRate] = kupiecTest_original(returns, VaR, q, leftTail)
    if nargin < 4, leftTail = false; end
    if leftTail
        hits = returns < -VaR;
    else
        hits = returns > VaR;
    end
    T = numel(hits);
    nHits = sum(hits);
    hitRate = nHits / T;
    p0 = 1 - q;
    epsVal = 1e-12;
    p0 = max(min(p0,1-epsVal), epsVal);
    hitRateAdj = max(min(hitRate,1-epsVal), epsVal);
    L0 = nHits*log(p0) + (T-nHits)*log(1-p0);
    L1 = nHits*log(hitRateAdj) + (T-nHits)*log(1-hitRateAdj);
    LRstat = -2*(L0 - L1);
    pValue = 1 - chi2cdf(LRstat,1);
end
```

Reconstruct_VAR_ES

```
function EVT_VaR_ES = Reconstruct_EVT_VaR_ES(AssetName, EVT_Results,
ConfidenceLevels)

    EVT_table = EVT_Results.OutTable;
    nCL = length(ConfidenceLevels);

    % Prepara tabella vuota
    EVT_VaR_ES = table('Size',[1, nCL*2], 'VariableTypes',
repmat("double",1,nCL*2));
    varNames = cell(1,nCL*2);
```

```
    for k = 1:nCL
        q = ConfidenceLevels(k);
        % Trova il threshold EVT più vicino
        [~, idx] = min(abs(EVT_table.ThresholdLevel - q));

        % Nomi colonne
        varNames{(k-1)*2+1} = ['VaR_' num2str(round(q*100))];
        varNames{(k-1)*2+2} = ['ES_'  num2str(round(q*100))];

        % Prendi i valori e correggi il segno (perdite negative)
        EVT_VaR_ES{1,(k-1)*2+1} = -EVT_table.VaR(idx);
        EVT_VaR_ES{1,(k-1)*2+2} = -EVT_table.CVaR(idx);
    end

    EVT_VaR_ES.Properties.VariableNames = varNames;
    EVT_VaR_ES.Properties.RowNames = {AssetName};
end
```

Plot EVT results

```
function analyzeEVTPlots_Pro(residStd, EVT_Results, outputDir, VaR_levels)
% ANALYZEEVTPLOTS_PRO
% Genera grafici di validazione per EVT (POT) usando residui standardizzati da
AR-GARCH
% Input:
%   residStd     = residui standardizzati (i.i.d. approx.)
%   EVT_Results  = struct con OutTable e Rolling (output EVT-POT)
%   outputDir    = cartella dove salvare i grafici
%   VaR_levels   = livelli di confidenza per il backtesting VaR (es. [0.95 0.97
0.99])

out = EVT_Results.OutTable;
thLevels = out.ThresholdLevel;
nLevels = numel(thLevels);

% Create output directory
if ~exist(outputDir,'dir'), mkdir(outputDir); end

%% 1. Return Level Plot
fig1 = figure('Visible','off');
plot(thLevels, out.VaR, 'o-','LineWidth',1.5);
title('Return Level Plot'); xlabel('Quantile Threshold'); ylabel('Return Level
(VaR)'); grid on;
saveas(fig1, fullfile(outputDir, 'Return_Level_Plot.png'));

%% 2. Mean Excess Plot
fig2 = figure('Visible','off');
```

```
thr = linspace(min(residStd), max(residStd), 100);
mean_excess = arrayfun(@(u) mean(residStd(residStd>u)-u), thr);
plot(thr, mean_excess, 'LineWidth',1.5);
title('Mean Excess Plot'); xlabel('Threshold u'); ylabel('Mean Excess'); grid on;
saveas(fig2, fullfile(outputDir, 'Mean_Excess_Plot.png'));

%% 3. PP-plot and QQ-plot for each threshold
for i = 1:nLevels
    q = thLevels(i);
    u = out.Threshold_u(i);
    xi = out.xi(i);
    sigma = out.sigma(i);
    exceed = residStd(residStd>u) - u;
    if isempty(exceed), continue; end

    % PP-plot
    figPP = figure('Visible','off');
    [F_emp, x_emp] = ecdf(exceed);
    F_fit = gp_cdf(x_emp, xi, sigma, 0);
    plot(F_emp, F_fit, 'o'); hold on;
    plot([0,1],[0,1],'r--');
    title(sprintf('PP-plot GPD q=%.2f',q)); xlabel('Empirical CDF'); ylabel('Fitted
GPD CDF'); grid on;
    saveas(figPP, fullfile(outputDir, sprintf('PP_plot_q%.2f.png',q)));

    % QQ-plot
    figQQ = figure('Visible','off');
    sims = gprnd(xi, sigma, 0, size(exceed));
    qqplot(exceed, sims);
    title(sprintf('QQ-plot Exceedances q=%.2f',q)); grid on;
    saveas(figQQ, fullfile(outputDir, sprintf('QQ_plot_q%.2f.png',q)));
end

%% 4. Tail Plot (log-log)
figTail = figure('Visible','off');
[F_emp, x_emp] = ecdf(residStd);    % calcola ECDF
plot(x_emp, 1-F_emp,'b-','LineWidth',1.5); % survival function (1-F)
set(gca,'XScale','log','YScale','log');
title('Tail Plot (log-log)');
xlabel('Residuals'); ylabel('1 - Empirical CDF');
grid on;
saveas(figTail, fullfile(outputDir, 'Tail_Plot.png'));

%% 5. Rolling xi and sigma
if isfield(EVT_Results, 'Rolling')
    fig4 = figure('Visible','off');
    subplot(2,1,1);
    plot(EVT_Results.Rolling.Time, EVT_Results.Rolling.xi,'LineWidth',1.5);
    title('Rolling \xi'); xlabel('Window Index'); ylabel('\xi'); grid on;
    subplot(2,1,2);
```

```
    plot(EVT_Results.Rolling.Time, EVT_Results.Rolling.sigma,'LineWidth',1.5);
    title('Rolling \sigma'); xlabel('Window Index'); ylabel('\sigma'); grid on;
    saveas(fig4, fullfile(outputDir, 'Rolling_Params.png'));
end

%% 6. Backtesting VaR
fig5 = figure('Visible','off');
plot(residStd,'k'); hold on; grid on;
for v = VaR_levels
    VaR_thr = quantile(residStd, 1-v);
    plot([1,length(residStd)], [VaR_thr, VaR_thr],'--','DisplayName',sprintf('VaR
%.2f',v));
    plot(find(residStd<VaR_thr), residStd(residStd<VaR_thr),'ro','MarkerSize',4);
end
title('Backtesting VaR on Standardized Residuals');
xlabel('Time'); ylabel('Residuals'); legend('show');
saveas(fig5, fullfile(outputDir, 'Backtesting_VaR.png'));

end
```

BACKTESTING FUNCTION

```
function [BacktestResults, VaR_ES_Table, RealResidues, CI_Low, CI_High,
DiagnosticsTable] = backtestingMonteCarloDiagnostics(stdResiduals, Dates,
rollingWindow, nSim, confidenceLevels, startDate, endDate, thresholdLevel, OutDir)

    %% ------------------------ Default parameters ------------------------
    if nargin < 6 || isempty(startDate), startDate = Dates(1); end
    if nargin < 7 || isempty(endDate),   endDate   = Dates(end); end
    if nargin < 8 || isempty(thresholdLevel), thresholdLevel = 0.95; end
    if nargin < 9 || isempty(OutDir), OutDir = 'Backtest_Diagnostics'; end

    %% ------------------------ Filter dates ------------------------------
    mask = Dates >= startDate & Dates <= endDate;
    stdResiduals = stdResiduals(mask);
    Dates = Dates(mask);
    nObs = numel(stdResiduals);
    nTests = nObs - rollingWindow;
    if nTests <= 0, error('Rolling window troppo grande rispetto ai dati'); end
    nLevels = numel(confidenceLevels);

    %% ------------------------ Preallocations ----------------------------
    VaR_sim = zeros(nTests, nLevels);
    ES_sim  = zeros(nTests, nLevels);
    RealResidues = zeros(nTests, 1);
    ViolationMatrix = false(nTests, nLevels);

    %% ------------------------ Main rolling loop -------------------------
```

```
    for t = 1:nTests
        win = stdResiduals(t:t+rollingWindow-1);
        mu_w = mean(win);
        sigma_w = std(win);

        % Threshold and exceedances
        u = quantile(win, thresholdLevel);
        exc = win(win>u) - u;

        if numel(exc)<10
            warning('Pochi exceedances al passo %d',t); continue;
        end

        % Fit GPD
        try
            ph = gpfit(exc);
            xi = ph(1); beta = ph(2);
        catch
            warning('Errore nel fitting GPD al passo %d',t); continue;
        end

        % Monte Carlo: mix normale + GPD
        nGPD = round(nSim*(1-thresholdLevel));
        nNormal = nSim - nGPD;
        sim_all = [normrnd(mu_w,sigma_w,[nNormal,1]); u + gprnd(xi,beta,0,
[nGPD,1])];

        % VaR / ES per ciascun livello
        for j = 1:nLevels
            cl = confidenceLevels(j);
            VaR_j = -quantile(sim_all, 1-cl);
            loss_j = sim_all(sim_all <= -VaR_j);
            if isempty(loss_j), ES_j = VaR_j; else ES_j = -mean(loss_j); end
            VaR_sim(t,j) = VaR_j;
            ES_sim(t,j)  = ES_j;
        end

        % Residuo reale
        RealResidues(t) = stdResiduals(t+rollingWindow);

        % Violazioni
        rl = -RealResidues(t);
        for j = 1:nLevels
            if rl > VaR_sim(t,j)
                ViolationMatrix(t,j) = true;
            end
        end
    end

    %% ----------------------- Summary statistics ----------------------
```

```
    Violations = sum(ViolationMatrix,1);
    MeanVaR    = mean(VaR_sim,1);
    MeanES     = mean(ES_sim,1);
    violationRate = Violations / nTests;

    % Clopper-Pearson 95% CI
    alpha = 0.05;
    CI_Low  = zeros(1,nLevels);
    CI_High = zeros(1,nLevels);
    for j = 1:nLevels
        k = Violations(j); n = nTests;
        CI_Low(j)  = betainv(alpha/2, k, n-k+1);
        CI_High(j) = betainv(1-alpha/2, k+1, n-k);
    end

    % Build BacktestResults table
    BacktestResults = table(confidenceLevels(:), MeanVaR(:), MeanES(:),
violationRate(:), CI_Low(:), CI_High(:), ...
                            'VariableNames',
{'ConfidenceLevel','MeanVaR','MeanES','ViolationRate','CI_Low','CI_High'});

    %% ------------------------ VaR/ES  ------------------------
    VaR_ES_Table = table(confidenceLevels(:), mean(VaR_sim,1)', mean(ES_sim,1)', ...
                    'VariableNames', {'ConfidenceLevel','MeanVaR','MeanES'});


    %% ------------------------ Diagnostics Table ------------------------
    Diagnostics = struct();
    Kupiec_LR = zeros(nLevels,1); Kupiec_p = zeros(nLevels,1); Kupiec_OK =
false(nLevels,1);
    Christ_LR = zeros(nLevels,1); Christ_p = zeros(nLevels,1); Christ_OK =
false(nLevels,1);

    for j = 1:nLevels
        V = ViolationMatrix(:,j);
        Nviol = sum(V);
        p0 = 1 - confidenceLevels(j);
        T = nTests;
        phat = Nviol / T;

        % Kupiec
        if phat>0 && phat<1
            LR_uc = -2*( (T-Nviol)*log((1-p0)/(1-phat)) + Nviol*log(p0/phat) );
            p_kupiec = 1 - chi2cdf(LR_uc,1);
        else
            LR_uc = NaN; p_kupiec = NaN;
        end
        Kupiec_LR(j) = LR_uc; Kupiec_p(j) = p_kupiec; Kupiec_OK(j) = p_kupiec>0.05;

        % Christoffersen
```

```
        Vnum = double(V);
        N00 = sum(Vnum(1:end-1)==0 & Vnum(2:end)==0);
        N01 = sum(Vnum(1:end-1)==0 & Vnum(2:end)==1);
        N10 = sum(Vnum(1:end-1)==1 & Vnum(2:end)==0);
        N11 = sum(Vnum(1:end-1)==1 & Vnum(2:end)==1);
        pi01 = N01/(N00+N01+eps);
        pi11 = N11/(N10+N11+eps);
        pi = (N01+N11)/(N00+N01+N10+N11+eps);
        logL_pooled = (N00+N01)*log(1-pi+eps) + (N10+N11)*log(pi+eps);
        logL_cond = N00*log(1-pi01+eps)+N01*log(pi01+eps)+N10*log(1-pi11+eps)
+N11*log(pi11+eps);
        LR_cc = -2*(logL_pooled - logL_cond);
        p_christ = 1 - chi2cdf(LR_cc,1);
        Christ_LR(j) = LR_cc; Christ_p(j) = p_christ; Christ_OK(j) = p_christ>0.05;
    end

    DiagnosticsTable = table(confidenceLevels(:), Kupiec_LR, Kupiec_p, Kupiec_OK,
Christ_LR, Christ_p, Christ_OK, ...
                             'VariableNames',
{'ConfidenceLevel','Kupiec_LR','Kupiec_pValue','Kupiec_ModelAccepted','Christ_LR','C
hrist_pValue','Christ_ModelAccepted'});

    %% ------------------------ Display tables ---------------------------
    disp('=== Backtest Summary ===');
    disp(BacktestResults);
    disp('=== Diagnostics Tests ===');
    disp(DiagnosticsTable);
    disp('=== VaR / ES Medi per Livello di Confidenza ===');
    disp(VaR_ES_Table);



    %% ------------------------ Save plots in OutDir ----------------------
    if ~exist(OutDir,'dir'), mkdir(OutDir); end
    timeVec = Dates(rollingWindow+1:end);

    % Residui vs VaR / ES
    fig1 = figure('Visible','off'); hold on;
    plot(timeVec, RealResidues,'k','DisplayName','Real Residuals');
    colors = lines(nLevels);
    for j = 1:nLevels
        plot(timeVec,
-VaR_sim(:,j),'--','Color',colors(j,:),'DisplayName',sprintf('VaR
%.2f',confidenceLevels(j)));
        plot(timeVec,
-ES_sim(:,j),':','Color',colors(j,:),'DisplayName',sprintf('ES
%.2f',confidenceLevels(j)));
    end
    title('Residuals vs Simulated VaR/ES'); xlabel('Date'); ylabel('Return / Risk
Threshold'); legend('Location','best'); grid on;
```

```
    saveas(fig1, fullfile(OutDir,'Residuals_vs_VaR_ES.png'));

    % Cumulative violations
    fig2 = figure('Visible','off'); hold on;
    for j = 1:nLevels
        cumViol = cumsum(ViolationMatrix(:,j));
        stairs(timeVec, cumViol,'DisplayName',sprintf('Cum Viol
%.2f',confidenceLevels(j)));
        yline((1-
confidenceLevels(j))*(1:nTests),'--','DisplayName',sprintf('Expected
%.2f',confidenceLevels(j)));
    end
    title('Cumulative Exceptions'); xlabel('Date'); ylabel('Cumulative
Violations'); legend('Location','best'); grid on;
    saveas(fig2, fullfile(OutDir,'Cumulative_Exceptions.png'));

end
```

Input of the Model

```
% Upload the dataset:
Dataset = readtable('commodities_data.xlsx',ImportErrorRule="omitrow");
```

```
Warning: Column headers from the file were modified to make them valid MATLAB identifiers before creating
variable names for the table. The original column headers are saved in the VariableDescriptions property.
Set 'VariableNamingRule' to 'preserve' to use the original column headers as table variable names.
```

```
if isempty(Dataset)
    error('Dataset is empty. Please check the file path or data format.');
end

% Take the Dates and the Prices:
Dates = Dataset{:,1};
Prices = Dataset{:,2:end};
if ~isa(Dates, 'datetime')
    Dates = datetime(Dates, 'ConvertFrom', 'excel');
end
if any(isnan(Prices), 'all')
    warning('Missing data detected in the Prices matrix.');
end
% Handle Missing Values (using linear interpolation)
Prices = fillmissing(Prices, 'linear');
% Ensure that Prices has no NaN rows
if any(isnan(Prices), 'all')
    warning('Missing data detected in the Prices matrix.');
end
```

```
% Get the Names of each assets
Names=Dataset.Properties.VariableNames(2:end);
```

ANALYSIS COMMODITIES

GOLD ASSET ANALYSIS

```
% Input
AssetName = Names{1,1};
Prices_GC_F = Prices(:, 1);
LogReturns_GC_F = diff(log(Prices_GC_F));
Dates_GC_F = Dates(2:end);
```

```
% Risk measures
calculateRiskStats(LogReturns_GC_F, AssetName)
```

```
    Asset Name    Number_of_Returns    Mean (%)    StdDev (%)    Variance (%)    Kurtosis    Skewness    VaR_95
    __________    _________________    ________    __________    ____________    ________    ________    ______

     {'GC_F'}           6491            -0.0371       1.0445         0.0109        8.8309      0.3139     -1.6809
ans = 1×13 table
                                                                                                                ...
```

	Asset Name	Number_of_Returns	Mean (%)	StdDev (%)	Variance (%)
1	'GC_F'	6491	-0.0371	1.0445	0.0109

```
generatePlotsForAsset(LogReturns_GC_F, AssetName);
```

```
% iid Proprierties Test (Autocorrelation and Normality) - (Ljung-Box and D'Agostino-Pearson)
[autocorrResults, turningPointResults] = testIIDProperties(LogReturns_GC_F, AssetName)
```

```
Warning: P is less than the smallest tabulated value, returning 0.001.
            LjungBox_pValue    LjungBox_QStat    LjungBox_CritVal    Normality_pValue    JarqueBera_pValue    Autoco
            _______________    ______________    ________________    ________________    _________________    ______

    GC_F        0.04399            18.718             18.307                0                  0.001             {'A

            TurningPoints    ExpectedTurningPoints    ZScoreTurningPoints    PValueTurningPoints    IsIID    IIDConc
            _____________    _____________________    ___________________    ___________________    _____    _______

    GC_F        4373                3245                  33.21                   0              false    {'Not
autocorrResults = 1×8 table
                                                                                                          ...
```

	LjungBox_pValue	LjungBox_QStat	LjungBox_CritVal	Normality_pValue
1 GC_F	0.04399	18.718	18.307	0

```
turningPointResults = 1×6 table
                                                                                ...
```

	TurningPoints	ExpectedTurningPoints	ZScoreTurningPoints
1 GC_F	4373	3245	33.21

```
plotACFandPACF(LogReturns_GC_F, AssetName, 20)
```

```
% AR-GARCH
[ADFResults, AR_GARCHResults, stdResiduals, ResidualDiagnostics,
AdditionalDiagnostics, EstModel, condVar] =
AR_GARCH_Model_SingleAsset_tStudents(LogReturns_GC_F, AssetName)
```

```
Warning: ADF critical value not available, using default -2.86
Warning: P is less than the smallest tabulated value, returning 0.001.
ADFResults = 1×4 table
```

	pValue	TestStatistic	CriticalValue_5%	StationarityConclusion
1 GC_F	0.001	-81.492	-2.86	"Stationary"

```
AR_GARCHResults = 1×14 table
```

...

	AssetName	AR1	ARCH1	GARCH1	Omega	Constant	DoF
1	'GC_F'	-0.01787	0.04785	0.94107	0	-0.00043	5.42

```
stdResiduals = 6491×1
      -1.3601
      -3.0592
    -0.011862
       1.5205
       2.0401
      0.50931
     -0.46006
      0.27525
     -0.93671
     -0.70009
        ⋮
ResidualDiagnostics = 1×4 table
```

...

	LjungBox_Resids_pVal	LjungBox_ResidsSq_pVal	TurningPoint_pVal
1 GC_F	0.40485	0.9986	0.33126

```
AdditionalDiagnostics = 1×5 table
```

...

	ARCH_pValue	ARCH_TestStatistic	ARCH_CriticalValue_5%	JB_pValue
1 GC_F	0.96502	0.96904	11.07	0.001

```
EstModel = 1×6 table
```

	VaR_95	CVaR_95	VaR_97	CVaR_97	VaR_99	CVaR_99
1 GC_F	-1.1272	-1.1663	-1.3571	-1.1379	-1.8821	-1.0244

```
condVar =
  arima with properties:
```

```
     Description: "ARIMA(1,0,0) Model (t Distribution)"
      SeriesName: "Y"
    Distribution: Name = "t", DoF = 5.41727
               P: 1
               D: 0
               Q: 0
        Constant: -0.000433021
              AR: {-0.0178741} at lag [1]
             SAR: {}
              MA: {}
             SMA: {}
     Seasonality: 0
            Beta: [1×0]
        Variance: [GARCH(1,1) Model]
```

```
% ME PLOT before EVT
plotMeanExcess(stdResiduals, AssetName, 100);
```

```
% threshold selection manually
outDir = 'THRESHOLD';
thresholds = 90:1:99;
p = [0.95, 0.975, 0.99];
leftTail = true;
threshold_selection(stdResiduals, AssetName, thresholds, p, outDir, leftTail);
```

```
% EVT
threshold=[0.96, 0.97];
opts.leftTail = true;
EVT_Results=EVT_POT_Model_Simplified(stdResiduals, AssetName, threshold, opts)
```

```
Risultati EVT POT per asset: GC_F
```

ThresholdLevel	Threshold_u	NumExceedances	xi	sigma	VaR	CVaR	AD_stat	CvM_stat
0.96	1.6728	260	0.13547	0.45123	1.6734	2.1952	0.21845	0.02842
0.97	1.7976	195	0.10766	0.49147	1.7983	2.349	0.26892	0.03744

```
EVT_Results = struct with fields:
    OutTable: [2×14 table]
```

```
% Reconstruct VaR and ES
CI = [0.95, 0.975, 0.99];
EVT_VaR_ES = Reconstruct_EVT_VaR_ES(AssetName, EVT_Results, CI);
disp(EVT_VaR_ES);
```

	VaR_95	ES_95	VaR_98	ES_98	VaR_99	ES_99
GC_F	-1.6734	-2.1952	-1.7983	-2.349	-1.7983	-2.349

```
% PLOT EVT results
outputDir = 'results_gold_EVT';
VaR_levels = [0.95 0.97 0.99];
analyzeEVTPlots_Pro(stdResiduals, EVT_Results, outputDir, VaR_levels);
```

Warning: Graphics timeout occurred. To share details of this issue with MathWorks technical support, please include that this is an unresponsive graphics client with your service request.

```
% BACKTESTING:  CRISIS 2008
rollingWindow = 500;
nSimulations = 100000;
confidenceLevels = [0.95,0.975, 0.99];

% Crisis 2008
startDate2008 = datetime(2007,1,1);
endDate2008   = datetime(2009,12,31);
% Filter
idx2008 = (Dates_GC_F >= startDate2008) & (Dates_GC_F <= endDate2008);
Dates_2008      = Dates_GC_F(idx2008);
stdResids_2008  = stdResiduals(idx2008);

%% Parametri opzionali
ThresholdLevel   = 0.96;
OutDir="Backtest_Crisis_2008_GOLD";

[BacktestResults_2008, VaR_sim_2008, ES_sim_2008, RealResidues_2008, CI_Low_2008,
CI_High_2008] = ...
    backtestingMonteCarloDiagnostics(stdResids_2008, Dates_2008, rollingWindow,
nSimulations, confidenceLevels, ...
                                     startDate2008, endDate2008,
ThresholdLevel,OutDir);
```

```
=== Backtest Summary ===
    ConfidenceLevel    MeanVaR    MeanES    ViolationRate      CI_Low      CI_High
    _______________    _______    ______    _____________    _________    ________

          0.95         1.7629     2.2129       0.042254        0.022021    0.072645
         0.975         2.1031     2.5087       0.014085       0.0038505    0.035667
          0.99         2.4961     2.8599      0.0070423      0.00085399    0.025206

=== Diagnostics Tests ===
    ConfidenceLevel    Kupiec_LR    Kupiec_pValue    Kupiec_ModelAccepted    Christ_LR    Christ_pValue    Christ_Mo
    _______________    _________    _____________    ____________________    _________    _____________    _________

          0.95          0.37784        0.53876              true                1.0631        0.30251             tr
         0.975           1.6442        0.19975              true                0.1147        0.73486             tr
          0.99          0.27988        0.59678              true               0.02847        0.86601             tr

=== VaR / ES Medi per Livello di Confidenza ===
    ConfidenceLevel    MeanVaR    MeanES
    _______________    _______    ______

          0.95         1.7629     2.2129
         0.975         2.1031     2.5087
          0.99         2.4961     2.8599
Warning: Limiting legend entries to 50. Specify a vector of graphics objects to display more than 50 entries.
```

```
% BACKTESTING COVID-19
rollingWindow2 = 250;
nSimulations2 = 100000;
confidenceLevels2 = [0.95,0.97, 0.99];

% Covid-19
startDateCOVID = datetime(2020,2,1);
endDateCOVID = datetime(2021,7,31);
% Filter
idxCOVID = (Dates_GC_F >= startDateCOVID) & (Dates_GC_F <= endDateCOVID);
Dates_COVID      = Dates_GC_F(idxCOVID);
stdResids_COVID  = stdResiduals(idxCOVID);

%% Parametri opzionali
ThresholdLevel   = 0.96;
OutDir           = 'Backtesting_Covid_GOLD';

[BacktestResults_COVID, VaR_sim_COVID, ES_sim_COVID, RealResidues_COVID,
CI_Low_COVID, CI_High_COVID] = ...
    backtestingMonteCarloDiagnostics(stdResids_COVID, Dates_COVID, rollingWindow2,
nSimulations2, confidenceLevels2, ...
    startDateCOVID, endDateCOVID, ThresholdLevel,OutDir);
```

```
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
```

```
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
```

```
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
=== Backtest Summary ===
    ConfidenceLevel    MeanVaR    MeanES    ViolationRate      CI_Low      CI_High
    _______________    _______    ______    _____________    _________    ________

         0.95           1.701     2.1512        0.05036       0.020484     0.10101
         0.97          1.9553     2.3725       0.035971       0.011781    0.081947
         0.99          2.4362        2.8      0.0071942      0.00018213    0.039431

=== Diagnostics Tests ===
    ConfidenceLevel    Kupiec_LR     Kupiec_pValue    Kupiec_ModelAccepted    Christ_LR    Christ_pValue    Christ_M
    _______________    _________     _____________    ____________________    _________    _____________    ________

         0.95          0.00037779       0.98449              true                0.9138       0.33911              t
         0.97             0.16034       0.68884              true                2.0798       0.14926              t
         0.99              0.1225       0.72634              true              0.014599       0.90383              t

=== VaR / ES Medi per Livello di Confidenza ===
    ConfidenceLevel    MeanVaR    MeanES
    _______________    _______    ______

         0.95           1.701     2.1512
         0.97          1.9553     2.3725
         0.99          2.4362        2.8
Warning: Limiting legend entries to 50. Specify a vector of graphics objects to display more than 50 entries.
```

```
% BACKTESTING Energy Crisis (2022)

rollingWindow3 = 250;
nSimulations3 = 100000;
confidenceLevels3 = [0.95,0.97, 0.99];

% Covid-19
startDateEnergyCrisis = datetime(2021,8,1);
endDateEnergyCrisis   = datetime(2022,12,31);
% Filter
idxEnergyCrisis = (Dates_GC_F >= startDateEnergyCrisis) & (Dates_GC_F <=
endDateEnergyCrisis);
Dates_EnergyCrisis      = Dates_GC_F(idxEnergyCrisis);
stdResids_EnergyCrisis  = stdResiduals(idxEnergyCrisis);
```

```
%% Parametri opzionali
ThresholdLevel   = 0.96;
OutDir           = 'Backtest_Energy_GOLD';



[BacktestResults_Energy, VaR_sim_Energy, ES_sim_Energy, RealResidues_Energy,
CI_Low_Energy, CI_High_Energy] = ...
    backtestingMonteCarloDiagnostics(stdResids_EnergyCrisis, Dates_EnergyCrisis,
rollingWindow3, nSimulations3, confidenceLevels3, ...
    startDateEnergyCrisis, endDateEnergyCrisis, ThresholdLevel,OutDir);
```

```
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
```

```
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
```

```
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
Warning: Maximum likelihood has converged to a boundary point of the parameter space.
Confidence intervals and standard errors can not be computed reliably.
=== Backtest Summary ===
```

ConfidenceLevel	MeanVaR	MeanES	ViolationRate	CI_Low	CI_High
0.95	1.4891	1.8944	0.067227	0.029469	0.12817
0.97	1.7186	2.0933	0.033613	0.0092331	0.083829
0.99	2.1496	2.4768	0.016807	0.0020419	0.059393

```
=== Diagnostics Tests ===
```

ConfidenceLevel	Kupiec_LR	Kupiec_pValue	Kupiec_ModelAccepted	Christ_LR	Christ_pValue	Christ_Mo
0.95	0.67421	0.41159	true	1.1647	0.2805	tr
0.97	0.051434	0.82059	true	0.28076	0.5962	tr
0.99	0.46236	0.49652	true	0.068969	0.79284	tr

```
=== VaR / ES Medi per Livello di Confidenza ===
```

ConfidenceLevel	MeanVaR	MeanES

```
    ____________    _______    ______

        0.95          1.4891     1.8944
        0.97          1.7186     2.0933
        0.99          2.1496     2.4768
Warning: Limiting legend entries to 50. Specify a vector of graphics objects to display more than 50 entries.
```

ANALYSIS BRENT

```
% Input
AssetName_BRENT = Names{1,2};
Prices_BRENT = Prices(:, 2);
LogReturns_BRENT = diff(log(Prices_BRENT));
Dates_BRENT = Dates(2:end);
```

```
% Risk measures
calculateRiskStats(LogReturns_BRENT, AssetName_BRENT)
```

```
      Asset Name        Number_of_Returns    Mean (%)    StdDev (%)    Variance (%)    Kurtosis    Skewness    VaR
    _______________    _________________    ________    __________    ____________    ________    ________    ___

    {'BrentCrudeOil'}          6491           -0.0147       2.2991         0.0529        13.464      0.6202     -3.7
ans = 1×13 table
                                                                                                              ...
```

	Asset Name	Number_of_Returns	Mean (%)	StdDev (%)	Variance (%)
1	'BrentCrudeOil'	6491	-0.0147	2.2991	0.0529

```
% iid Proprierties Test (Autocorrelation and Normality) - (Ljung-Box and D'Agostino-
Pearson)
[autocorrResults_BRENT, turningPointResults_BRENT] =
testIIDProperties(LogReturns_BRENT, AssetName_BRENT)
```

```
Warning: P is less than the smallest tabulated value, returning 0.001.
                    LjungBox_pValue    LjungBox_QStat    LjungBox_CritVal    Normality_pValue    JarqueBera_pValue
                    _______________    ______________    ________________    ________________    _________________

    BrentCrudeOil       0.091052           16.312              18.307                0                  0.001

                    TurningPoints    ExpectedTurningPoints    ZScoreTurningPoints    PValueTurningPoints    IsIID
                    _____________    _____________________    ___________________    ___________________    _____

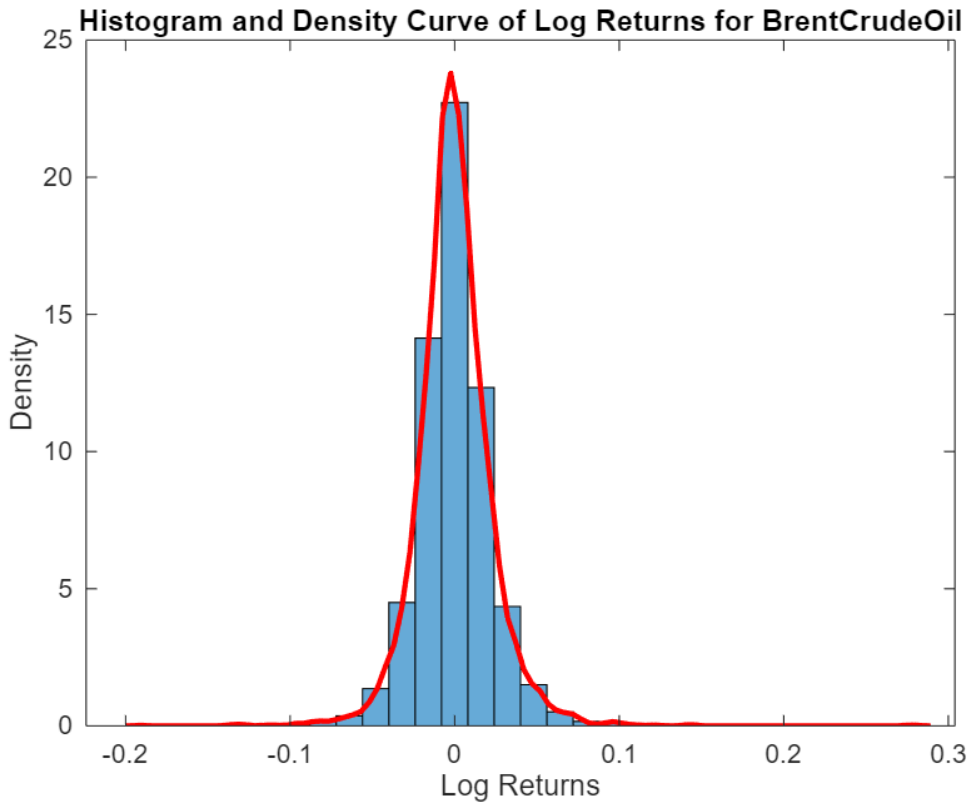
    BrentCrudeOil         4363                3245                 32.916                   0             false
autocorrResults_BRENT = 1×8 table
                                                                                                              ..
```

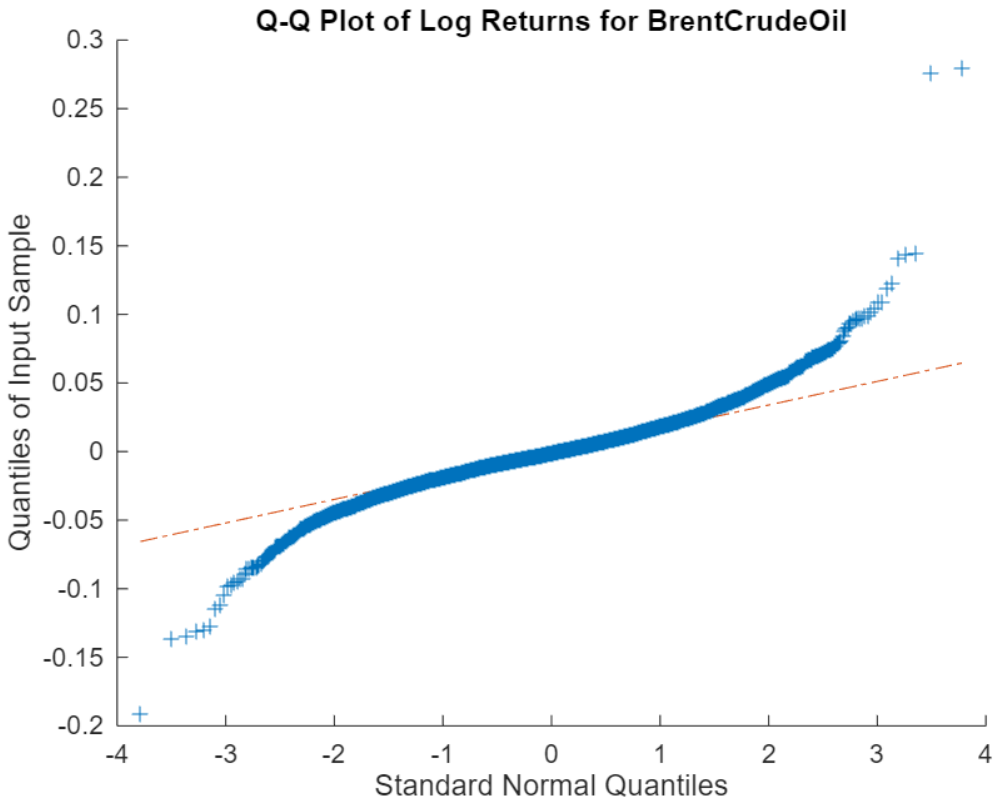
	LjungBox_pValue	LjungBox_QStat	LjungBox_CritVal	Normality_pValue
1 BrentCrudeOil	0.091052	16.312	18.307	0

```
turningPointResults_BRENT = 1×6 table
                                                                                                          ...
```

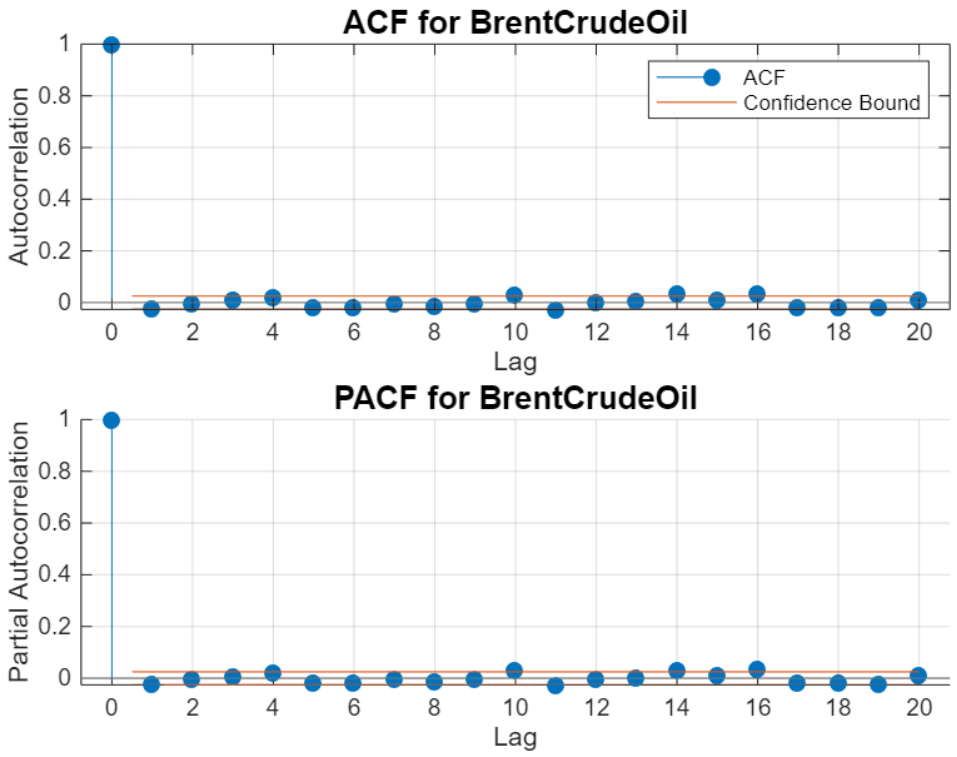
	TurningPoints	ExpectedTurningPoints	ZScoreTurningPoints
1 BrentCrudeOil	4363	3245	32.916

```
generatePlotsForAsset(LogReturns_BRENT, AssetName_BRENT);
```






```
plotACFandPACF(LogReturns_BRENT, AssetName_BRENT, 20)
```




```
% AR-GARCH
[ADFResults_BRENT, AR_GARCHResults_BRENT, stdResiduals_BRENT,
ResidualDiagnostics_BRENT, AdditionalDiagnostics_BRENT, EstModel_BRENT,
condVar_BRENT] = AR_GARCH_Model_SingleAsset_tStudents(LogReturns_BRENT,
AssetName_BRENT)
```

```
Warning: ADF critical value not available, using default -2.86
Warning: P is less than the smallest tabulated value, returning 0.001.
ADFResults_BRENT = 1×4 table
```

	pValue	TestStatistic	CriticalValue_5%	StationarityConclusion
1 BrentCrudeOil	0.001	-82.244	-2.86	"Stationary"

```
AR_GARCHResults_BRENT = 1×14 table
```

...

	AssetName	AR1	ARCH1	GARCH1	Omega	Constant	DoF
1	'BrentCrudeOil'	-0.02793	0.08451	0.90572	1e-05	-0.00077	7.17

```
stdResiduals_BRENT = 6491×1
         1.482
       -1.6451
       0.83111
        0.8814
        2.7133
        2.2007
    -0.098047
       0.11485
      -0.42918
       0.18248
```

⋮

ResidualDiagnostics_BRENT = 1×4 table

...

	LjungBox_Resids_pVal	LjungBox_ResidsSq_pVal	TurningPoint_pVal
1 BrentCrudeOil	0.68567	0.84452	0.92962

AdditionalDiagnostics_BRENT = 1×5 table

...

	ARCH_pValue	ARCH_TestStatistic	ARCH_CriticalValue_5%	JB_pValue
1 BrentCrudeOil	0.81903	2.2125	11.07	0.001

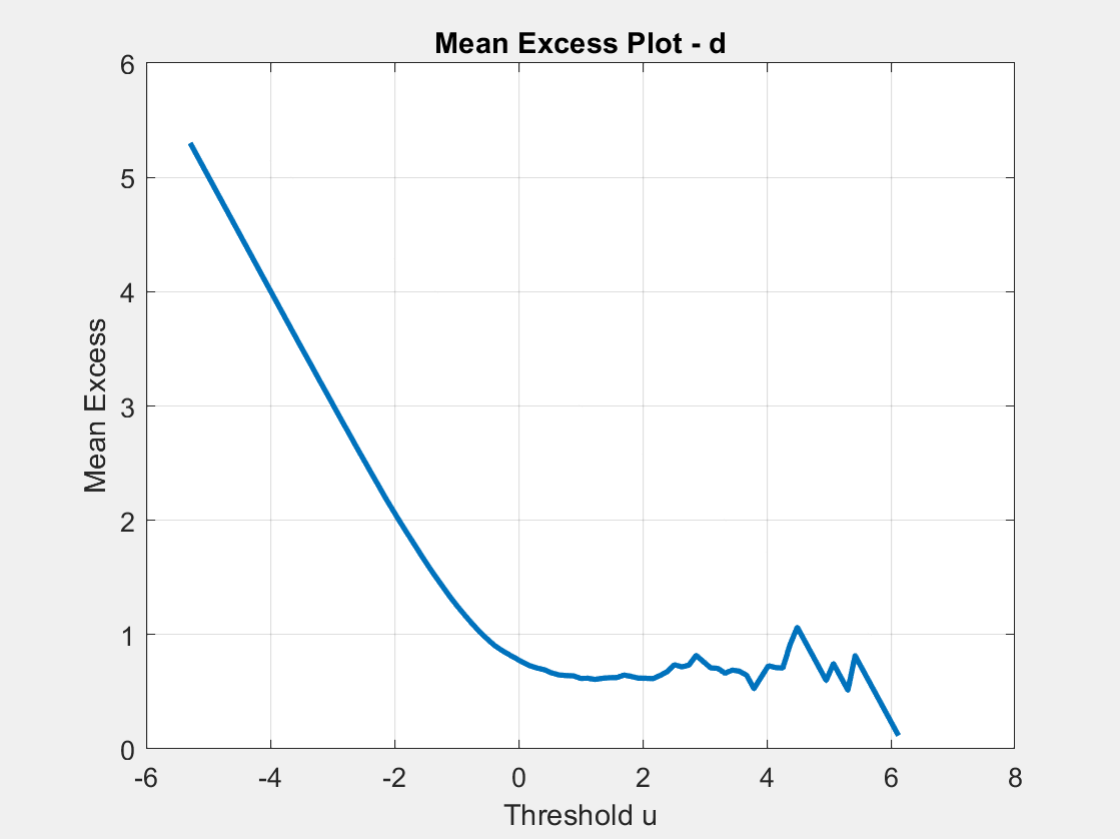
EstModel_BRENT = 1×6 table

	VaR_95	CVaR_95	VaR_97	CVaR_97	VaR_99	CVaR_99
1 BrentCrudeOil	-3.1186	-3.3571	-3.6994	-3.3765	-4.9647	-3.2506

```
condVar_BRENT =
  arima with properties:

     Description: "ARIMA(1,0,0) Model (t Distribution)"
      SeriesName: "Y"
    Distribution: Name = "t", DoF = 7.16787
               P: 1
               D: 0
               Q: 0
        Constant: -0.000771491
              AR: {-0.0279262} at lag [1]
             SAR: {}
              MA: {}
             SMA: {}
     Seasonality: 0
            Beta: [1×0]
        Variance: [GARCH(1,1) Model]
```

```
% ME PLOT
plotMeanExcess(stdResiduals_BRENT, AssetName_BRENT, 100);
```




```
Mean Excess plot salvato in: BrentCrudeOil\MeanExcess_d.png
```

```
% Threshold selection
outDir = 'THRESHOLD';
thresholds = 90:1:99;
p = [0.95, 0.975, 0.99];
leftTail = true;
threshold_selection(stdResiduals_BRENT, AssetName_BRENT, thresholds, p, outDir,
leftTail);
```

```
 Grafici salvati in: THRESHOLD
```

```
% EVT
threshold_BRENT=[0.95, 0.97];
opts.leftTail = true;
EVT_Results_BRENT=EVT_POT_Model_Simplified(stdResiduals_BRENT, AssetName_BRENT,
threshold_BRENT, opts)
```

```
Risultati EVT POT per asset: BrentCrudeOil
    ThresholdLevel    Threshold_u    NumExceedances        xi          sigma      VaR      CVaR     AD_stat    CvM_s
    ______________    ___________    ______________    _________    _______    ______    ______    _______    _____

         0.95            1.6106            325          1.9934e-05    0.50146    1.6113    2.1127    0.49417    0.070
         0.97            1.8726            195            0.037397    0.47295    1.8732    2.3645    0.56481    0.099
EVT_Results_BRENT = struct with fields:
    OutTable: [2×14 table]
```

```
EVT_OutTable=EVT_Results.OutTable;
```

```
% Reconstruct VaR and ES
threshold_BRENT = [0.95, 0.97, 0.99];
EVT_VaR_ES = Reconstruct_EVT_VaR_ES(AssetName_BRENT, EVT_Results_BRENT,
threshold_BRENT);
disp(EVT_VaR_ES);
```

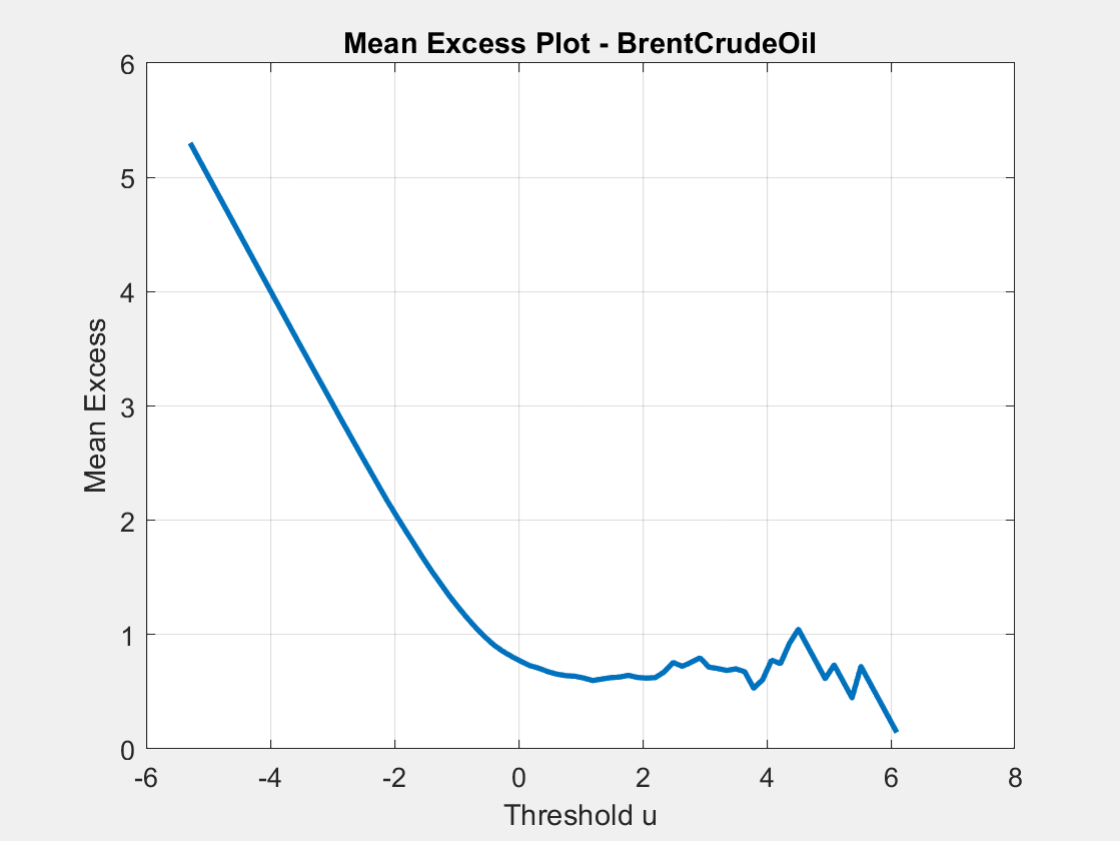
	VaR_95	ES_95	VaR_97	ES_97	VaR_99	ES_99
BrentCrudeOil	-1.6113	-2.1127	-1.8732	-2.3645	-1.8732	-2.3645

```
% PLOT EVT results

outputDir = 'results_brent_EVT';
VaR_levels = [0.95 0.97 0.99];
analyzeEVTPlots_Pro(stdResiduals_BRENT, EVT_Results_BRENT, outputDir, VaR_levels);
```

```
Warning: Negative data ignored
```

```
% ME PLOT
plotMeanExcess(stdResiduals_BRENT, 'plots_Gold', 'BrentCrudeOil',nPoints);
```




```
Mean Excess plot salvato in: plots_Gold\MeanExcess_BrentCrudeOil.png
```

```
% BACKTESTING:  CRISIS 2008
rollingWindow_BRENT = 250;
```

```
nSimulations_BRENT = 1000;
confidenceLevels_BRENT = [0.95, 0.975, 0.99];

% 2008 Crisis
startDate2008_BRENT = datetime(2007,1,1);
endDate2008_BRENT   = datetime(2009,12,31);
% Filter
idx2008_BRENT = (Dates_BRENT>= startDate2008_BRENT) & (Dates_BRENT <=
endDate2008_BRENT);
Dates_2008_BRENT      = Dates_BRENT(idx2008_BRENT);
stdResids_2008_BRENT  = stdResiduals_BRENT(idx2008_BRENT);

%% Parametri opzionali
ThresholdLevel_BRENT   = 0.95;
OutDir_BRENT           = 'Backtest_Crisis_2008_BRENT';


[BacktestResults_BRENT_2008, VaR_sim_BRENT_2008, ES_sim_BRENT_2008,
RealResidues_BRENT_2008, CI_Low_BRENT_2008, CI_High_BRENT_2008] = ...
    backtestingMonteCarloDiagnostics(stdResids_2008_BRENT, Dates_2008_BRENT,
rollingWindow_BRENT, nSimulations_BRENT, confidenceLevels_BRENT, ...
    startDate2008_BRENT, endDate2008_BRENT, ThresholdLevel_BRENT,OutDir_BRENT);
```

```
=== Backtest Summary ===
    ConfidenceLevel    MeanVaR    MeanES    ViolationRate      CI_Low      CI_High
    _______________    _______    ______    _____________    _________    ________

          0.95         1.6342     2.0569       0.052434       0.035121    0.074893
         0.975         1.9553     2.3355       0.022472       0.011664    0.038925
          0.99         2.3251     2.6624      0.0074906      0.0020446    0.019067

=== Diagnostics Tests ===
    ConfidenceLevel    Kupiec_LR    Kupiec_pValue    Kupiec_ModelAccepted    Christ_LR    Christ_pValue    Christ_Mo
    _______________    _________    _____________    ____________________    _________    _____________    _________

          0.95         0.065627        0.79781              true                3.1065       0.077978             tr
         0.975          0.14486        0.70349              true               0.55283        0.45716             tr
          0.99          0.37194        0.54195              true              0.060492        0.80572             tr

=== VaR / ES Medi per Livello di Confidenza ===
    ConfidenceLevel    MeanVaR    MeanES
    _______________    _______    ______

          0.95         1.6342     2.0569
         0.975         1.9553     2.3355
          0.99         2.3251     2.6624
Warning: Limiting legend entries to 50. Specify a vector of graphics objects to display more than 50 entries.
```

```
% BACKTESTING:  COVID
rollingWindow2_BRENT = 250;
nSimulations2_BRENT = 1000;
confidenceLevels2_BRENT = [0.95, 0.975, 0.99];
```

```
% covid
startDateCOVID_BRENT = datetime(2020,2,1);
endDateCOVID_BRENT = datetime(2021,7,31);
% Filter
idxCOVID_BRENT = (Dates_BRENT >= startDateCOVID_BRENT) & (Dates_BRENT <=
endDateCOVID_BRENT);
Dates_COVID_BRENT      = Dates_BRENT(idxCOVID_BRENT);
stdResids_COVID_BRENT  = stdResiduals_BRENT(idxCOVID_BRENT);

%% Parametri opzionali
ThresholdLevel_BRENT   = 0.95;
OutDir_BRENT           = 'Backtesting_Covid_BRENT';


% APPLICATION
[BacktestResults_BRENT_Covid, VaR_sim_BRENT_Covid, ES_sim_BRENT_Covid,
RealResidues_BRENT_Covid, CI_Low_BRENT_Covid, CI_High_BRENT_Covid] = ...
    backtestingMonteCarloDiagnostics(stdResids_COVID_BRENT, Dates_COVID_BRENT,
rollingWindow2_BRENT, nSimulations2_BRENT, confidenceLevels2_BRENT, ...
    startDateCOVID_BRENT, endDateCOVID_BRENT, ThresholdLevel_BRENT,OutDir_BRENT);
```

```
=== Backtest Summary ===
    ConfidenceLevel    MeanVaR    MeanES    ViolationRate      CI_Low      CI_High
    _______________    _______    ______    _____________    _________    ________

          0.95         1.8391      2.292       0.064748        0.030032     0.11935
         0.975          2.179     2.5913       0.035971        0.011781    0.081947
          0.99         2.5879     2.9356       0.014388       0.0017473    0.051008

=== Diagnostics Tests ===
    ConfidenceLevel    Kupiec_LR    Kupiec_pValue    Kupiec_ModelAccepted    Christ_LR    Christ_pValue    Christ_Mo
    _______________    _________    _____________    ____________________    _________    _____________    _________

          0.95          0.58468        0.44448              true                1.2568        0.26225             tr
         0.975          0.60566        0.43643              true               0.37603        0.53974             tr
          0.99          0.23808         0.6256              true              0.058826        0.80836             tr

=== VaR / ES Medi per Livello di Confidenza ===
    ConfidenceLevel    MeanVaR    MeanES
    _______________    _______    ______

          0.95         1.8391      2.292
         0.975          2.179     2.5913
          0.99         2.5879     2.9356
Warning: Limiting legend entries to 50. Specify a vector of graphics objects to display more than 50 entries.
```

```
% BACKTESTING:  Energy Crisis(2022)
rollingWindow3_BRENT = 250;
nSimulations3_BRENT = 1000;
confidenceLevels3_BRENT = [0.95, 0.975, 0.99];

% Energy Crisis(2022)
startDateEnergyCrisis_BRENT = datetime(2021,8,1);
endDateEnergyCrisis_BRENT   = datetime(2022,12,31);
% Filter
```

```
idxEnergyCrisis_BRENT = (Dates_BRENT >= startDateEnergyCrisis_BRENT) & (Dates_BRENT
<= endDateEnergyCrisis_BRENT);
Dates_EnergyCrisis_BRENT      = Dates_BRENT(idxEnergyCrisis_BRENT);
stdResids_EnergyCrisis_BRENT  = stdResiduals_BRENT(idxEnergyCrisis_BRENT);

%% Parametri opzionali
ThresholdLevel_BRENT   = 0.94;
OutDir_BRENT           = 'Backtesting_Ukraine_BRENT';

% APPLICATION
[BacktestResults_BRENT_Energy, VaR_sim_BRENT_Energy, ES_sim_BRENT_Energy,
RealResidues_BRENT_Energy, CI_Low_BRENT_Energy, CI_High_BRENT_Energy] = ...
    backtestingMonteCarloDiagnostics(stdResids_EnergyCrisis_BRENT,
Dates_EnergyCrisis_BRENT, rollingWindow3_BRENT, nSimulations3_BRENT,
confidenceLevels3_BRENT, ...
    startDateEnergyCrisis_BRENT, endDateEnergyCrisis_BRENT,
ThresholdLevel_BRENT,OutDir_BRENT);
```

```
=== Backtest Summary ===
    ConfidenceLevel    MeanVaR    MeanES    ViolationRate     CI_Low      CI_High
    _______________    _______    ______    _____________    _________    ________

          0.95         1.6943     2.1396       0.05042        0.018726     0.10652
         0.975         2.0367     2.4279      0.033613       0.0092331    0.083829
          0.99         2.4202     2.7603      0.016807       0.0020419    0.059393

=== Diagnostics Tests ===
    ConfidenceLevel    Kupiec_LR     Kupiec_pValue    Kupiec_ModelAccepted    Christ_LR    Christ_pValue    Christ_M
    _______________    _________    _____________    ____________________    _________    _____________    ________

          0.95         0.00044111       0.98324              true                1.166        0.28023              t
         0.975            0.32748       0.56714              true              0.28076         0.5962              t
          0.99            0.46236       0.49652              true             0.068969        0.79284              t

=== VaR / ES Medi per Livello di Confidenza ===
    ConfidenceLevel    MeanVaR    MeanES
    _______________    _______    ______

          0.95         1.6943     2.1396
         0.975         2.0367     2.4279
          0.99         2.4202     2.7603
Warning: Limiting legend entries to 50. Specify a vector of graphics objects to display more than 50 entries.
```